OWASP Top 10 Vulnerabilities

OWASP Top 10 2025:

1. A01:2025- Broken Access Control
2. A02:2025- Security Misconfiguration
3. A03:2025- Software Supply Chain Failure
4. A04:2025- Cryptographic Failure
5. A05:2025- Injection
6. A06:2025- Insecure Design
7. A07:2025- Authentication Failure
8. A08:2025- Software or Data Integrity Failure
9. A09:2025- Security Logging & Alerting Failure
10. A10:2025- Mishandling of Exceptional Conditions

A01:2025- Broken Access Control:

What it is:

- Broken access controls are a type of security vulnerability that arises when an application or system fails to properly restrict access to sensitive data or functionality.
- This vulnerability allows attackers to gain unauthorized access to resources that should be restricted, such as user accounts, files, databases, or administrative functions.
- Broken access controls can occur due to a variety of factors, including poor design, configuration errors, or coding mistakes.

What is Access Control?

A security mechanism used to control which **users or systems are allowed to access a particular resource or system**. Access control is implemented in computer systems to ensure that **only authorized users have access to resources**, such as files, directories, databases, and web pages. The primary goal of access control is to protect sensitive data and ensure that it is only accessible to those who are authorized to access it

How an attacker could exploit it:

- The most common technique is **manipulating an identifier in a URL or API request** : for

example, changing /invoice/1024 to /invoice/1025 to view someone else's invoice (an Insecure Direct Object Reference).

- Other tactics include forcing a browser to hit an admin-only URL directly, tampering with a JWT or cookie to escalate privileges, or bypassing client-side "you can't do this" checks since those checks were never enforced on the server.

Real-world example:

Instagram Private Posts (API Endpoint Bypass)

In 2019, a vulnerability was discovered in Instagram's API. The application checked privacy flags on the client side, but the backend API failed to enforce these checks. Attackers exploiting this flaw could manipulate API requests to view private photos and stories of other users without authorization

Mitigation Technique:

Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata. Following are some of the ways to prevent broken access control:

1. **Implement Role-Based Access Control (RBAC):**

RBAC is a method of regulating access to computer or network resources based on the roles of individual users within an enterprise. By defining roles in an organization and assigning access rights to these roles, we can control what actions a user can perform on a system.

2. **Use Parameterized Queries:**

Parameterized queries are a way to protect PHP applications from SQL Injection attacks, where malicious users could potentially gain unauthorized access to your database. By using placeholders instead of directly including user input into the SQL query, we can significantly reduce the risk of SQL Injection attacks.

3. **Proper Session Management:**

Proper session management ensures that authenticated users have timely and appropriate access to resources, thereby reducing the risk of unauthorized access to sensitive information. Session management includes using secure cookies, setting session timeouts, and limiting the number of active sessions a user can have.

4. **Use Secure Coding Practices:**

Secure coding practices involve methods to prevent the introduction of security vulnerabilities. Developers should **sanitize and validate user input** to prevent malicious data from causing harm and avoid using insecure functions or libraries.

A03:2025- Software Supply Chain Failure

What it is:

- Software supply chain failures refers to when vulnerabilities or malicious code enter a system via external dependencies, tools, or build processes, rather than in your own code.
- Modern softwares are not written from scratch, they are assembled from open-source packages, third-party libraries, CI/CD pipelines, and build tools. So, when any link in this chain is compromised i.e a malicious package, a hijacked maintainer account, or a tampered build process, It can cause a supply chain attack

How an attacker could exploit it:

- **Compromised Dependencies:** Threat actors upload malicious packages to public repositories (like npm or PyPI) using typosquatting or by taking over abandoned projects.
- **Dependency Confusion**: Attackers publish public packages with the same names as internal, private corporate libraries. Automated build systems often pull the public versions by mistake.
- **CI/CD Pipeline Intrusions:** Attackers compromise developer tools, IDE extensions, or build scripts, inserting malicious code directly into the software during compilation or testing.
- **Implicit Trust in Auto-Updates:** Relying on automatic updates or fetching resources from external CDNs without verifying integrity allows compromised vendor updates to infiltrate secure environments.
- **Vulnerable & Outdated Components:** High Impact CVEs in common libraries (e.g. Log4j) , create massive security risks across countless apps.

Real-world example:

SolarWinds Orion attack in 2020:

One of the famous real world example for software supply chain failure is the SolarWinds Orion attack in 2020. Attackers breached the vendor's build pipeline and inserted malicious code into legitimate product updates. When 18,000 customers blindly installed this routine update, it created a massive back door for attackers to infiltrate Fortune 500 companies and US federal agencies

Mitigation Technique:

Securing the software supply chain requires rigorous tracking and verification across all stages of development:

- **Maintain an SBOM:** Create and continuously update a Software Bill of Materials (SBOM) to track all direct and transitive dependencies.
- **Use SCA Tools:** Utilize Software Composition Analysis (SCA) to automatically scan

your code and identify outdated or vulnerable libraries.

- **Secure CI/CD Pipelines:** Restrict access using the principle of least privilege, avoid storing plain-text secrets, and secure build environments against unauthorized changes.
- **Verify Components:** Avoid blind auto-updates. Pin dependencies to specific versions and check cryptographic hashes or digital signatures before pulling in external code.

A04:2025- Cryptographic Failure:

What it is:

Cryptographic Failures occur when applications fail to properly protect sensitive data using encryption. This vulnerability allows attackers to read, modify, or steal sensitive information such as passwords, credit card details, and personal data.

- Happens due to weak, missing, or incorrect encryption
- Includes plaintext data storage, broken algorithms, and poor key management
- Often caused by misuse of cryptographic libraries or outdated standards
- Can lead to data breaches, identity theft, and financial loss

How an attacker could exploit it:

Attackers target weak encryption, poor key handling, and unprotected data flows.

- **Plaintext Data Access:**

Attackers read sensitive data stored without encryption. i.e. Database breach reveals plaintext passwords.

- **Weak Hash Cracking:**

Attackers crack weak password hashes using precomputed tables, i.e. MD5 password hashes cracked in seconds.

- **Man-in-the-Middle Attacks**

Attackers intercept unencrypted network traffic, i.e. Credentials stolen on public Wi-Fi due to missing HTTPS.

- **Key Theft**

Attackers steal hardcoded or exposed encryption keys, i.e. API key found in source code repository.

- **Cryptographic Downgrade Attacks**

Attackers force the use of weak encryption protocols, i.e. TLS downgraded to insecure SSL version.

Real-world example:

The **2013 Adobe data breach** is a prime example. Hackers accessed 150 million records because Adobe stored millions of passwords encrypted using weak, outdated hashing algorithms

(Triple DES and weak SHA-1). The failure to use secure algorithms made the data much easier to crack, leading to massive credential exposure.

Mitigation Technique:

Use Strong Encryption Standards:

- Use modern algorithms like AES-256 and SHA-256
- Avoid deprecated or broken cryptographic methods

Protect Sensitive Data

- Encrypt data at rest and in transit
- Never store passwords in plaintext (use strong hashing)

Secure Key Management

- Store keys securely (HSM, vaults)
- Rotate and protect encryption keys regularly

Enforce HTTPS Everywhere

- Use TLS for all communications
- Enable HSTS to prevent downgrade attacks

Use Trusted Cryptographic Libraries

- Avoid custom encryption logic
- Follow industry-approved cryptographic practices

A08:2025- Software or Data Integrity Failure:

What it is:

Software and Data Integrity Failures occur when applications rely on untrusted software, updates, libraries, or data without proper integrity verification. This vulnerability allows attackers to modify code, inject malicious updates, or compromise the software supply chain.

- Happens due to lack of integrity checks and verification mechanisms
- Includes tampered updates, malicious dependencies, and compromised CI/CD pipelines
- Often caused by blind trust in external components or automation
- Can lead to complete application compromise and data manipulation

How an attacker could exploit it:

Attackers target trust relationships between applications, updates, and dependencies.

- **Malicious Dependency Injection:**

Attackers publish malicious libraries that applications unknowingly trust, for example a CI pipeline installs a dependency from a public repository that contains backdoor code.

- **Tampered Software Updates:**

Attackers replace legitimate updates with malicious ones, for example a server updates deliver

malware because they were not digitally signed.

- **CI/CD Pipeline Manipulation:**

Attackers gain access to build or deployment systems. They can modify artifacts that are deployed directly to production.

- **Data Integrity Attacks:**

Critical data is altered without validation checks, i.e. configuration files or model data are modified to change application behavior.

- **Supply Chain Attacks:**

Attackers compromise a single vendor to infect many users, for example trusted library update contains malicious code affecting thousands of applications.

Real-world example:

In **2021, Codecov**, a software auditing platform, suffered a software and data integrity failure. Hackers exploited vulnerability in Codecov's Docker image creation process. They modified the widely used Bash Uploader script to secretly export customers' sensitive data (credentials and API keys) to an external server. Because the script was downloaded and run continuously in thousands of client CI/CD pipelines without integrity checks, the attackers successfully compromised the internal environments of hundreds of Codecov's customers**.**

Mitigation Technique:

Verify Software Integrity

- Use digital signatures and checksums
- Verify updates before installation
- Reject unsigned or tampered code

Secure Dependencies

- Use trusted repositories only
- Lock dependency versions
- Monitor for vulnerable or malicious packages

Secure CI/CD Pipelines

- Restrict access to build and deployment systems
- Use signed build artifacts
- Audit pipeline activities regularly

Implement Integrity Controls

- Validate critical data before processing
- Use hashing and verification mechanisms
- Monitor for unauthorized changes

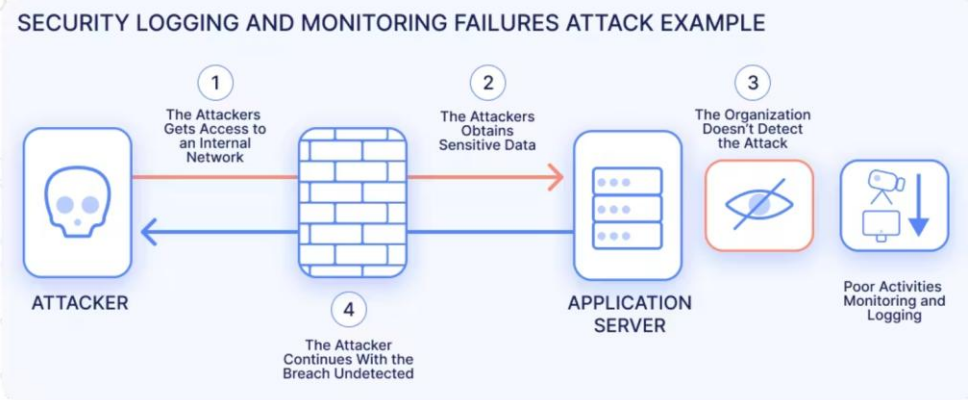
Continuous Monitoring & Auditing

- Track dependency changes
- Scan for supply chain vulnerabilities
- Perform regular security reviews

A09:2025- Security Logging & Alerting Failure:

What it is:

Security Logging & Alerting Failures refers to security weaknesses that occur when applications and systems do not properly record, monitor, or alert on security-relevant events. This means that critical actions such as failed login attempts, privilege misuse, data access, or system errors—are either not logged at all, logged incorrectly, or never reviewed. As a result, attacks can continue unnoticed for long periods, giving attackers time to move laterally, steal data, or cause damage without being detected. Effective logging, real-time alerting, and regular log analysis are essential to quickly identify suspicious activity, respond to incidents, and support forensic investigations.



How an attacker could exploit it:

Attackers exploit these failures primarily in three ways:

- **Unnoticed Brute Force & Credential Stuffing:** If an application fails to log failed login attempts or lacks lockout/rate-limiting policies, attackers can run automated dictionaries to guess passwords or "stuff" leaked credentials until they gain unauthorized access.
- **Stealthy Lateral Movement:** Once inside a network, adversaries typically escalate privileges and move between servers. Without real-time alerting on anomalous activities or geographic sign-in anomalies, attackers can dwell in systems and exfiltrate sensitive data for months without detection.
- **Tampering and Log Deletion**: Organizations that store logs locally without integrity protection allow attackers who gain root or admin access to easily edit, delete, or overwrite log files, erasing all evidence of their intrusion and making forensic investigations impossible

Real-world example:

Marriott/Starwood Data Breach (2018): Attackers gained unauthorized access to the Starwood guest reservation database. Because the company lacked centralized logging and automated alerts, the breach went entirely undetected for four years. This failure allowed the exfiltration of approximately 500 million customer records

Mitigation Technique:

- **Log Significant Security Events:** A web application should generate logs for certain security events, such as successful and failed logins or attempted access to sensitive resources.
- **Include Necessary Context:** Logs and alerts should include the context required to identify and respond to potential threats. For example, a log of a failed login attempt should include the account identifier to allow correlation of multiple failed attempts for a user account and detection of likely compromised user accounts.
- **Store Logs Securely:** Log files should be stored in a way that means they will likely be available for analysis in the wake of a cybersecurity incident. This means that an organization should establish log retention timelines and store logs on a secure system to prevent deletion by an attacker.
- **Protect Against Injection**: Log files may include user-provided data. This data should be encoded in a way that protects against injection attacks targeting log monitoring and alerting solutions.
- **Implement Monitoring Processes:** Logs only provide value if they are reviewed for signs of a potential attack. The organization should have processes in place to ensure that they can quickly identify and address potential intrusions.
- **Create an Incident Response Strategy:** In the event of a successful cyberattack, the security team needs to act swiftly to minimize the impacts of the attack. The organization should have an incident response plan in place and train responders to ensure a quick, correct response.

Personal Reflection:

Security Logging and Alerting Failures surprised me the most, because it's the only one on this list that isn't really a "bug" at all. It's an absence. The Equifax incident emphasized this point: the hackers didn't require a groundbreaking new exploit to remain undetected for 76 days; they simply relied on a company with an expired certificate that silently turned off its own visibility. It made me realize that a lot of "sophisticated" breaches are actually enabled by relatively simple operational weaknesses , which feels like an important mindset shift going into CTF prep — the forensics and detection side matters just as much as finding the flashy exploit.

References:

https://owasp.org/Top10/2025/

https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

https://tryhackme.com/room/owaspbrokenaccesscontrol

https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/

https://www.authgear.com/post/owasp-2025-software-supply-chain-failures/

https://www.geeksforgeeks.org/

https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/

https://owasp.org/Top10/2025/A08_2025-Software_or_Data_Integrity_Failures/

https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/